

The Evidence: What We Proved in Part 2

The Evidence: What We Proved in Part 2 I

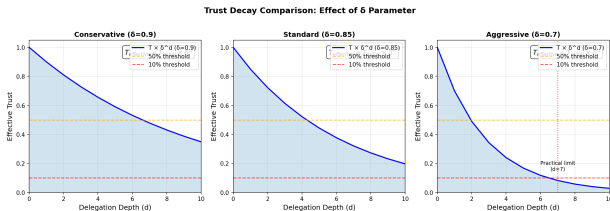


Figure 1: Effective trust vs. delegation depth at decay factors $\delta = \{0.9, 0.85, 0.7\}$, with 50% and 10% thresholds marking delegation bounds.

The Evidence: What We Proved in Part 2 I

Part 1 supplies the formal apparatus. Part 2 supplies the empirical evaluation: tested CIF modules, a **950-attack** corpus, and an architecture-aware simulation over four headline topologies (Claude Code, AutoGPT, CrewAI, LangGraph), with broader adapter coverage documented in Part 2.

Here is what the data says.

The Experimental Setup

The Experimental Setup I

We tested four common multi-agent architectures found in production:

The Experimental Setup I

- ▶ **Claude Code** (Hierarchical)
- ▶ **AutoGPT** (Autonomous Loop)
- ▶ **CrewAI** (Role-Based Team)
- ▶ **LangGraph** (State Machine)

The Experimental Setup I

The test corpus included direct prompt injection, poisoned RAG contexts, deep trust exploitation, and multi-turn social engineering.

Finding 1: Defense Layering vs. Individual Efficacy

Finding 1: Defense Layering vs. Individual Efficacy I

The Data: Individual defenses (like just a firewall) stopped ~60–70% of attacks in the parametric evaluation. The full CIF stack achieved a **96–100% parametric detection ceiling**, with direct injection detected at 99–100% across architectures. The separate real-pipeline evaluation had a lower multi-seed mean of approximately 44.8%. **The Implication:** The defenses demonstrated orthogonal coverage. The firewall blocked inputs that the sandbox would have missed, and the sandbox identified anomalies that the trust calculus would have permitted. The data suggests that removing any single layer creates a statistically significant vulnerability gap.

Finding 2: State Machine Determinism

Finding 2: State Machine Determinism I

The Data: **LangGraph** architectures achieved the highest detection rates (98%). **The Mechanism:** In a state machine, valid transitions are explicitly defined. Attempts to move from an Analyze state to an Execute state without passing through Verify were caught immediately. **The Implication:** Explicit state definitions provide a “security for free” effect, where the architecture itself enforces behavioral invariants that are difficult to bypass.

Finding 3: Hierarchical Vulnerability

Finding 3: Hierarchical Vulnerability I

The Data: **Claude Code** and **AutoGPT** styles were strong against external attacks but vulnerable to **Systemic** (Ω_5) compromise. **The Mechanism:** When the “Boss” agent was tricked, it ordered “Worker” agents to execute malicious actions, and they complied. **The Implication:** Hierarchical systems exhibit a Single Point of Failure at the root. Security in these topologies requires worker-level tripwires that can reject orders even from authenticated superiors.

Finding 4: Roles as Security Boundaries

Finding 4: Roles as Security Boundaries I

The Data: CrewAI performed exceptionally well against privilege escalation. **The Mechanism:** “Roles” acted as effective containers for capabilities. A “Researcher” agent lacked the tools to write code, rendering code-execution attacks against it inert. **The Implication:** Strict role definitions function as effective security boundaries, limiting the blast radius of a compromised agent.

Finding 5: The Stealth-Impact Tradeoff

Finding 5: The Stealth-Impact Tradeoff I

The Data: High-impact attacks were consistently easier to detect than low-impact nudges. **The Implication:** “Catastrophic” takeover attempts generate significant noise in the system state. The primary challenge for defenders is not the sudden takeover, but the slow, subtle drift of agent beliefs over long time horizons.

Finding 6: Emergent Misalignment Is the Nash-Optimal Attack

Finding 6: Emergent Misalignment Is the Nash-Optimal Attack I

The colony benchmark reveals a striking pattern: **emergent misalignment achieves the lowest detection rate (74.3%) at the highest false positive rate (25.5%)** of any evaluated scenario. (These are Part 2's 30-seed benchmark means; an earlier single-seed figure of 56.1% is not the publication estimate.) Part 2's game-theoretic analysis explains why: in the zero-sum game between CIF and an adversary, emergent misalignment is the Nash-equilibrium attack strategy. It is the attacker's best response to full CIF deployment.

Finding 6: Emergent Misalignment Is the Nash-Optimal Attack I

The game-theoretic payoff matrix shows that: - Full CIF achieves 99–100% detection against direct injection (Ω_1) and 96–100% against impersonation, the corpus category standing for trust exploitation (Ω_4) - Against emergent misalignment (distributed sub-threshold drift with no explicit adversaries), detection falls to 74.3% - A rational adversary, knowing CIF is deployed, will prefer emergent misalignment over direct injection

Finding 6: Emergent Misalignment Is the Nash-Optimal Attack I

This is not a failure of CIF—it is a consequence of its success. When explicit attacks are reliably detected, adversaries are forced toward the subtlest and most distributed manipulation strategies. The 74.3% detection rate on emergent misalignment represents the current frontier of defensive capability, not a gap in the framework's design.

Operator implication: Deploy colony-scale entropy monitoring and schedule periodic manual behavioral audits (weekly for high-stakes deployments). The Ω_5 playbook (sec:incident-response) provides the response protocol when drift accumulates despite in-context detection.

Finding 7: The Implementation Gap Is a Feature, Not a Bug

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

The 51–88 percentage-point gap between the parametric ceiling (96–100%) and the empirical pipeline mean (44.8%) reflects **adapter implementation maturity**, not a failure of CIF's formal architecture. Part~2 introduces a 5-level CMMI-style adapter maturity scale:

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

Level	Name	Marginal TPR	Description
1	Stub	\$ \$0%	Hardcoded scores; no domain logic
2	Heuristic	1–5%	Pattern matching; uncalibrated thresholds
3	Statistical	5–15%	Calibrated thresholds; regression features
4	Adaptive	15–30%	Online learning; per-architecture tuning
5	Verified	30%+	Formal guarantees; cross-architecture validated

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

The rubric column is each adapter's *marginal* contribution to true-positive rate, not a whole-pipeline detection rate: a Level-5 adapter is one that adds 30 or more percentage points when introduced, not one that reaches 30% detection on its own.

The current Claude Code adapter is at Level 3 (Statistical), explaining the 44.8% mean. The roadmap projects +35–41 percentage points of improvement by advancing adapters to Level 5 for the primary attack categories. The parametric ceiling (96–100%) represents what Level-5 adapters achieve—it is a design target, not an overclaim.

Finding 7: The Implementation Gap Is a Feature, Not a Bug I

Operator implication: When deploying CIF, assess the maturity level of each adapter against your threat model. Level-3 adapters (current) provide meaningful protection against unsophisticated Ω_1 – Ω_2 attacks; Level-4–5 adapters (planned) are required for Ω_4 – Ω_5 protection. The gap is closeable—it is an engineering challenge, not a theoretical limitation.

A Note on the Numbers

A Note on the Numbers I

The detection rates in Part 2 are derived from a calibrated parametric simulation, modeled on the architecture's topology. They represent the *structural* security of the design.

A Note on the Numbers I

- ▶ **94% Overall Detection** means: “Across all architectures and attack categories, 94% of attack vectors are detected by the full CIF defense stack” (with architecture-specific rates ranging from 94–98%).
- ▶ It does **not** mean: “We have a magic Python script that catches 94% of all evil AI thoughts.”

A Note on the Numbers I

We proved the *architecture* works. The implementation fidelity is the variable for the builder.

A Note on the Numbers II

A Note on Three Numbers: Throughout this guide you will encounter three detection rates that may seem contradictory. They are not — they measure different things:

- ▶ **96–100%** (parametric simulation, $N = 3,800$): C1F's **design-level detection ceiling** — what the defense architecture achieves when adapters are fully mature (Level 5) and conditions match the calibrated model. This is the target, not the current reality.
- ▶ **44.8%** [95% HDI: 41.3%, 48.3%] (multi-seed pipeline, 30 seeds): The **current empirical baseline** for the Claude Code architecture with Level-3 adapters. This is what you get today, out of the box, before adapter tuning.
- ▶ **~12.2%** (ablation corpus, 98 attacks, all categories including hardest): The **conservative floor** — full pipeline performance on a corpus specifically designed to include difficult attacks. This represents the worst-case realistic estimate.

All three numbers are correct. Use 44.8% for realistic planning, 96% as the floor of the achievable ceiling with mature adapters, and ~12.2% as a conservative lower bound for adversarial threat modeling.

Tripwire Configuration Data I

The simulations utilized the following tripwire densities to achieve the reported results:

Tripwire Configuration Data I

Category	Count	Used in Sim	Placement Strategy
Identity canaries	3+	per agent	Core identity beliefs
Boundary canaries	5+	per agent	Permission boundaries
Principal canaries	2+	per agent	Trust relationships
Temporal canaries	1	per agent	Session continuity